Informe técnico – Práctica 2

Programación orientada a objetos en C++: modelo de regresión lineal

Lenguajes de Programación y Paradigmas – Universidad EAFIT

Dato	Información
Integrante 1	Simon Sanchez Marquez
Integrante 2	Manuel Garzón Valderrama
Integrante 3	Jorge Andrés Marin Hernandez
Repositorio	https://github.com/simonsannchez321-byte/practica2-sjm
Video YouTube	https://youtu.be/dzAhtJJJx0s?si=lqWS-kc8Lyh-QNyI

Este documento fue elaborado con ayuda de una herramienta de inteligencia artificial, que se usó para organizar y redactar el informe a partir de los resultados reales del programa. El código, las ejecuciones y las conclusiones corresponden al trabajo del grupo.

1. Objetivo y alcance

El objetivo fue construir en C++ una clase que ajusta modelos de regresión lineal simple y múltiple, sin usar ninguna librería externa. Por eso las operaciones con matrices y las estadísticas se programaron desde cero. Además se hizo un análisis de dónde vive cada objeto en la memoria y una investigación de tres fallos históricos causados por mal manejo de memoria dinámica.

Se usaron dos archivos de datos: Ice_cream_selling_data.csv (regresión simple, 49 filas) y student_exam_scores.csv (regresión múltiple, 200 filas).

2. Organización del código

Archivo	Para qué sirve
Matrix.hpp	Clase Matrix con transpuesta, producto y resolución de sistemas de ecuaciones.
Stats.hpp	Funciones de estadística: media, desviación estándar, MSE, MAE y R^2.
CsvReader.hpp	Lee un CSV y separa las variables de entrada (X) de la salida (y, última columna).
LinearRegression.hpp	La clase del modelo (atributos weights y bias, y los métodos pedidos).
main.cpp	Programa de pruebas: regresión simple, múltiple y un caso extra.
memory_analysis.cpp	Programa del análisis de memoria (direcciones y tamaños).

3. Documentación de los métodos

3.1 Clase LinearRegression

Atributos principales: weights (un peso por cada variable de entrada) y bias (el término independiente). También guarda means y stds (media y desviación de cada columna del entrenamiento, necesarias para escalar), y dos banderas: useScaling (si se escala o no) y fitted (si el modelo ya fue entrenado).

Método	Qué hace, entradas y salidas
LinearRegression(bool useScaling)	Constructor. Recibe si se debe escalar los datos. No recibe datos todavía.
fit(X, y)	Entrena el modelo. Entra la matriz X (filas = muestras) y el vector y. Escala X si corresponde, agrega una columna de unos para el sesgo y resuelve las ecuaciones normales $(A^TA)\beta = A^Ty$. El primer valor de β queda en bias y el resto en weights.
predict(X)	Devuelve un vector con la predicción de cada fila: bias + suma de peso × valor. Aplica el mismo escalado que se usó al entrenar.
score(X, y)	Devuelve $R^2 = 1 - SS_res / SS_tot$. Cerca de 1 es buen ajuste; cerca de 0 significa que el modelo explica casi nada.
mse(X, y) y mae(X, y)	Error cuadrático medio y error absoluto medio, para medir el error en las unidades de la variable de salida.

Método	Qué hace, entradas y salidas
data_scaling(X, fitParams)	Estandariza cada columna: z = (x − media) / desviación. Si fitParams es verdadero calcula y guarda la media y la desviación; si es falso reutiliza las del entrenamiento (así el test se escala igual que el train).
getWeights(), getBias()	Devuelven los parámetros en la escala estandarizada.
getWeightsOriginalScale(), getBiasOriginalScale()	Convierten los parámetros a las unidades originales para poder leer la ecuación (peso/desviación y sesgo ajustado por las medias).

3.2 Clase Matrix y funciones auxiliares

Método	Qué hace
transpose()	Devuelve la matriz transpuesta.
multiply(Matrix) y multiply(vector)	Producto matriz por matriz y matriz por vector. Lanza excepción si las dimensiones no coinciden.
solve(A, b) (estático)	Resuelve A·x = b con Gauss-Jordan y pivoteo parcial (elige la fila con el mayor valor en cada columna para evitar divisiones inestables). Lanza excepción si el pivote es casi cero, es decir, si la matriz es singular.
Stats::mean, stddev	Media y desviación estándar poblacional.
Stats::mse, mae, r2	Medidas de error y de ajuste; r2 devuelve 0 si todos los valores reales son iguales.
CsvReader::load(ruta)	Lee el archivo, quita el BOM de UTF-8 y los saltos \r si existen, y lanza excepción si una fila tiene un número de columnas distinto.

3.3 Partición de datos

La función split del programa principal manda cada quinta fila al conjunto de prueba y el resto al de entrenamiento, para tener 80 % y 20 %. Es determinista: siempre produce la misma partición, así los resultados se pueden repetir.

4. Evidencia de pruebas

Se compila con build.sh y se ejecuta main en la carpeta donde están los CSV. A continuación, entradas y salidas de cada prueba tomadas de la ejecución real.

4.1 Regresión simple: helados

Entrada: Temperature (°C) como única variable y Ice Cream Sales (units) como salida. 49 filas: 40 de entrenamiento y 9 de prueba.

```
weights (escalados): -2.098920
bias (escalado): 16.238461
Ice Cream Sales = 16.437667 - 0.769281*[Temperature]

score R2 (train): 0.028741
score R2 (test) : 0.014853
MSE (test)      : 116.565345
MAE (test)      : 9.793199
```

Predicciones sobre test (primeras 4 de 9):

Posición	y_hat (predicho)	real
1	19.190579	32.284531
2	18.478020	20.279679
3	17.714940	12.615181
4	16.552778	4.673643

Lectura del resultado: el R^2 es muy bajo, pero no es un error del programa. Los datos forman una parábola (las ventas son altas con temperaturas muy bajas y muy altas, y mínimas cerca de 0 °C), y una recta no puede seguir esa forma.

4.2 Regresión múltiple: notas de examen

Entrada: hours_studied, sleep_hours, attendance_percent y previous_scores como variables; exam_score como salida. 200 filas: 160 de entrenamiento y 40 de prueba.

```
weights (escalados): 4.885806 1.497372 1.526922 2.923723
bias (escalado): 34.387500
exam_score = -2.744329 + 1.535452*[hours_studied]
  + 1.025201*[sleep_hours] + 0.107781*[attendance_percent]
  + 0.183456*[previous_scores]

score R2 (train): 0.837611
score R2 (test) : 0.839650
MSE (test)      : 7.578097
MAE (test)      : 2.255676
```

Predicciones sobre test (primeras 4 de 40):

Posición	y_hat (predicho)	real
1	40.493733	40.300000
2	28.417787	24.700000
3	41.692551	41.100000
4	22.285017	17.100000

Las 40 predicciones completas están en el archivo salida_regresion.txt del repositorio. El R^2 de entrenamiento (0.838) y el de prueba (0.840) son casi iguales, lo que indica que el modelo no se sobreajustó.

4.3 Prueba adicional: helados con término cuadrático

Para comprobar la explicación de la parábola se agregó una segunda variable, la temperatura al cuadrado, y se volvió a entrenar con la misma clase.

```
Ice Cream Sales = 2.946690 - 0.785058*[Temperature] + 1.796630*[Temperature^2]
score R2 (train): 0.935833
score R2 (test) : 0.904218
MSE (test)      : 11.333218
MAE (test)      : 3.104096
```

El R^2 de prueba pasó de 0.015 a 0.904 y el MSE bajó de 116.6 a 11.3. Con esto queda confirmado que el problema era la forma de los datos y no el código.

5. Análisis de memoria

5.1 Direcciones y tamaños

Se creó un modelo en memoria global, otro local (pila) y otro dinámico (montículo o heap). Salida real de una ejecución (las direcciones cambian en cada corrida porque el sistema operativo las aleatoriza):

Objeto	Región	Dirección	sizeof
globalModel	Global (datos/BSS)	0x55ecff1d3160	88 bytes
localModel	Pila (stack)	0x7ffc67cb7200	88 bytes
new LinearRegression()	Heap	0x55ed2100b2b0	88 bytes

El puntero heapModel, que apunta al objeto del heap, vive en la pila (0x7ffc67cb7168) y ocupa 8 bytes. Se nota que las direcciones del global y del heap empiezan en 0x55…, mientras que la de la pila empieza en 0x7ff…, es decir, son zonas distintas de la memoria del programa.

5.2 ¿Por qué 88 bytes?

La clase tiene tres vectores de doubles (weights, means y stds), un double (bias) y dos bool. En un sistema de 64 bits con libstdc++, un std::vector ocupa 24 bytes (tres punteros: inicio, fin y fin de capacidad).

- 3 vectores × 24 bytes = 72 bytes.
- 1 double (bias) = 8 bytes, para un total de 80.
- 2 bool × 1 byte = 2 bytes, para un total de 82.
- Relleno (padding) hasta un múltiplo de 8, que es la alineación exigida por los punteros y el double: 82 pasa a 88.

El tamaño es el mismo en las tres regiones: la región solo decide dónde vive el objeto, no cuánto ocupa. Tampoco cambia después de fit(): los valores de los pesos están en un bloque aparte del heap (en la prueba, weights.data() apuntó a 0x55ed2100d090 con 3 elementos), y sizeof solo cuenta los tres punteros de control de cada vector.

5.3 Vector de modelos en memoria dinámica

Se creó new std::vector<LinearRegression>(5). Resultados reales:

- El objeto vector está en el heap (0x55ed2100d530) y mide 24 bytes.
- Su buffer de elementos está en otro bloque (0x55ed2100df90), con size = 5 y capacity = 5.
- Tamaño del buffer: 5 × 88 = 440 bytes.
- Los elementos son contiguos y se separan exactamente 0x58 = 88 bytes: …df90, …dfe8, …e040, …e098, …e0f0.

Un arreglo clásico new LinearRegression[5] también reserva 440 bytes para los objetos. En ambos casos no se cuenta la memoria que cada modelo reserve por dentro para sus propios vectores al entrenarse, ni la pequeña cabecera que el administrador de memoria pueda agregar.

6. Fallos históricos por manejo de memoria

6.1 Heartbleed (2014)

- **Involucrados:** el proyecto OpenSSL (versiones 1.0.1 a 1.0.1f). Lo descubrieron de forma independiente ingenieros de Codenomicon y Neel Mehta, de Google. La agencia tributaria de Canadá (CRA) fue una de las afectadas.
- **Consecuencias:** la extensión Heartbeat devolvía hasta 64 KB de memoria del servidor por cada petición (lectura fuera de los límites del buffer), lo que podía exponer claves privadas, contraseñas y cookies. En la CRA se robaron unos 900 números de seguro social.
- **Cómo se corrige:** comprobar que la longitud declarada en el mensaje no sea mayor que los datos realmente recibidos (arreglo en OpenSSL 1.0.1g), revocar y reemitir certificados y contraseñas, y usar herramientas como AddressSanitizer, Valgrind y fuzzing.
- **Fuentes:** https://heartbleed.com ; https://globalnews.ca/news/1269168/900-sin-numbers-stolen-due-to-heartbleed-bug-canada-revenue-agency

6.2 Gusano Morris (1988)

- **Involucrados:** Robert Tappan Morris, estudiante de posgrado de Cornell, que lo lanzó el 2 de noviembre de 1988. Afectó sistemas Unix de universidades, laboratorios y entidades del gobierno de Estados Unidos. Fue la primera condena bajo la ley Computer Fraud and Abuse Act.
- **Consecuencias:** se suele citar que infectó cerca de 6000 máquinas (cifra aproximada). Uno de sus caminos de ataque fue un desbordamiento de buffer en el servicio fingerd, causado por la función gets(), que no revisa el tamaño de lo que recibe. La corte estimó el costo de limpieza entre 200 y 53 000 dólares por instalación.
- **Cómo se corrige:** usar funciones con límite de tamaño (fgets, strncpy), validar el tamaño de los buffers y compilar con protecciones modernas como canarios de pila, NX y ASLR.
- **Fuentes:** https://en.wikipedia.org/wiki/Morris_worm ; E. Spafford, “The Internet Worm Program: An Analysis”, Purdue CSD-TR-823, https://spaf.cerias.purdue.edu/tech-reps/823.pdf

6.3 Cloudbleed (2017)

- **Involucrados:** la empresa Cloudflare (su analizador de HTML generado con Ragel). Lo reportó Tavis Ormandy, de Google Project Zero.
- **Consecuencias:** los servidores leían más allá del final de un buffer y devolvían memoria con cookies, tokens y datos de peticiones de otros clientes; parte quedó guardada en cachés de buscadores. El mayor impacto fue entre el 13 y el 18 de febrero de 2017, en alrededor de 1 de cada 3 300 000 peticiones. Se encontraron 770 URL en caché de 161 dominios y se purgaron junto con Google, Yahoo y Bing. Las claves privadas SSL no se filtraron.
- **Cómo se corrige:** la causa fue comprobar el final del buffer con == en lugar de >=, de modo que el puntero pudo pasarse del límite. Usar >= lo habría detectado. También se recomienda un analizador más simple y verificado, y fuzzing.
- **Fuente:** https://blog.cloudflare.com/incident-report-on-memory-leak-caused-by-cloudflare-parser-bug/

Los tres casos comparten la misma raíz: el programa leía o escribía fuera del espacio de memoria que realmente le pertenecía.

7. Problemas encontrados y soluciones

Problema	Solución
R^2 casi cero en la regresión de helados.	Se revisó la forma de los datos, se verificó con un cálculo independiente en Python y se agregó la variable x^2, con lo que R^2 subió a 0.90.
Matriz singular (columnas repetidas en una prueba de memoria).	Matrix::solve detecta un pivote casi cero y lanza una excepción en lugar de dar números sin sentido. Se corrigió usando columnas independientes.
El CSV traía BOM de UTF-8 y saltos \r\n, y stod fallaba.	El lector elimina el BOM y los \r antes de convertir los números.
En el entorno MSYS2 el ejecutable se cerraba por conflicto de librerías dinámicas.	Se compiló con la opción -static (ver build.sh).

8. Conclusiones

1. Separar el modelo, el álgebra de matrices y la estadística en clases distintas hizo el código más fácil de probar y de reutilizar.
2. Estandarizar las variables no cambia las predicciones de los mínimos cuadrados, pero mejora la estabilidad numérica y permite comparar los pesos entre sí: en el caso de las notas, hours_studied es la variable que más pesa (4.89 en escala estandarizada).
3. Un R^2 bajo no siempre significa que el código esté mal. En los helados mostró que una recta no sirve para datos con forma de parábola; hay que mirar los datos antes de elegir el modelo.
4. En las notas de examen el modelo generaliza bien: R^2 de 0.838 en entrenamiento y 0.840 en prueba, sin sobreajuste evidente.
5. sizeof de un objeto no incluye la memoria que sus miembros piden al heap. Entender pila, heap y memoria global ayuda a evitar fugas y accesos inválidos, que fueron la causa de los fallos históricos estudiados.
6. Las ecuaciones normales dan la solución exacta y bastan con pocas variables; si hubiera muchas, convendría un método como el descenso del gradiente.
7. Hacer las operaciones con matrices a mano ayudó a entender qué hay detrás de una librería de aprendizaje automático.

9. Video

Enlace al video YouTube: https://youtu.be/dzAhtJJJx0s?si=lqWS-kc8Lyh-QNyI